

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

**Лабораторная работа №1 по курсу**

**«Операционные системы»**

Группа: М8О-208БВ-25

Студент: Матвиенко М.В.

Преподаватель: Бахарев В.Д.

Оценка: \_\_\_\_\_

Дата: 26.09.26

Москва, 2026

# Постановка задачи

## Вариант 15.

### Цель работы

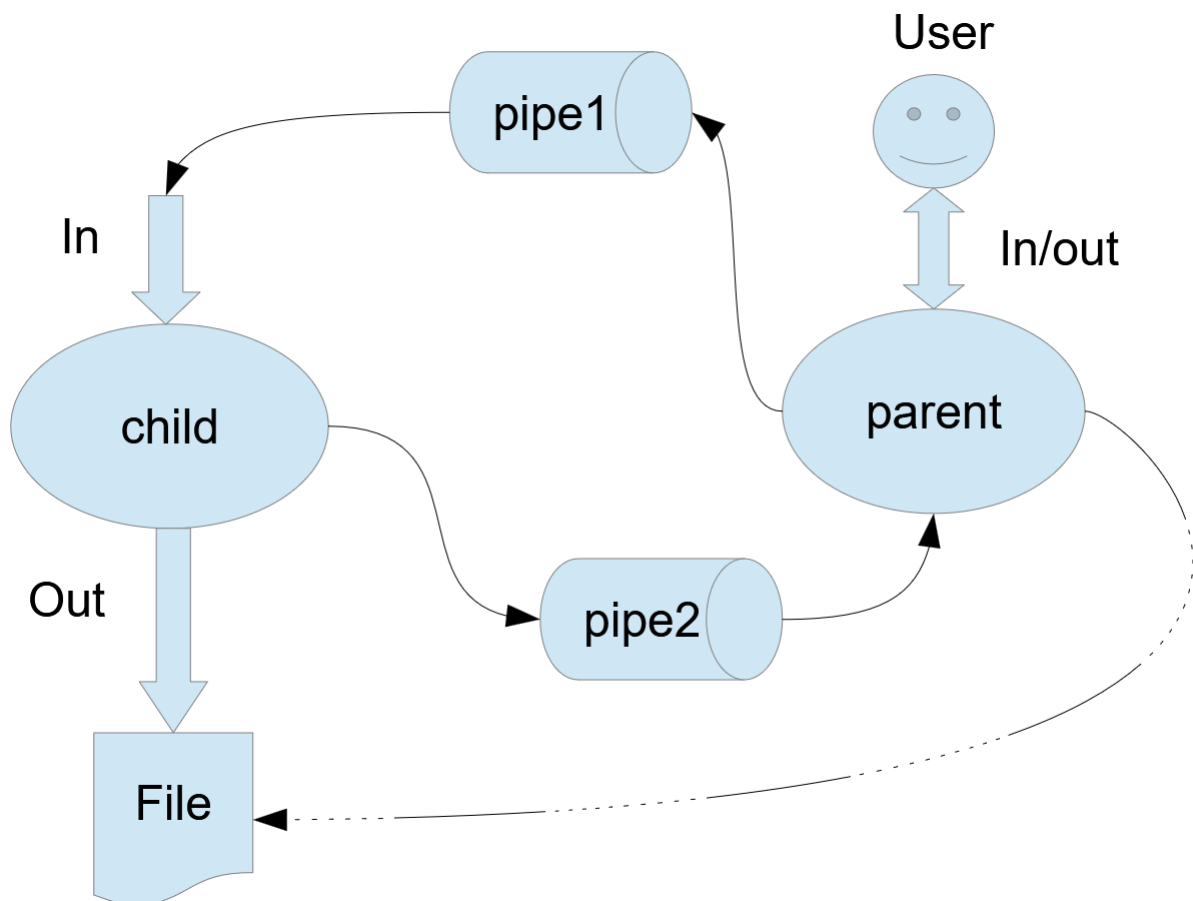
Приобретение практических навыков в:

- Управление процессами в ОС
- Обеспечение обмена данных между процессами посредством каналов

### Задание

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Схема организации межпроцессорного взаимодействия:



Родительский процесс создает дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись. Перенаправление стандартных потоков ввода-вывода показано на картинке выше. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child проверяет строки на валидность правилу. Если строка соответствует правилу, то она выводится

в стандартный поток вывода дочернего процесса, иначе в `pipe2` выводится информация об ошибке. Родительский процесс полученные от `child` ошибки выводит в стандартный поток вывода. Правило проверки для `child`: строка должна начинаться с заглавной буквы.

## Общий метод и алгоритм решения

Программа начинается с запуска кода родителя, то есть основного процесса, родитель читает отдельно первую строку (рассчитано на 256 знаков или меньше) и открывает файл для записи (файл ищется в той же директории, где лежит программа, если файла с таким именем нет, создаётся новый в той же директории, если такой файл существует, все данные в нём будут затёрты).

- `int file_fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);` - открытие файла.

После этого создаются две трубы: одна для передачи данных от родителя к ребёнку, другая для возврата ошибок.

- `pipe(pipe_data)` и `pipe(pipe_error)`.

Далее идёт создание дочернего процесса с помощью `fork()`:

- `pid_t child = fork();`

Теперь у нас есть два полностью одинаковых процесса, выполняющих код, идущий после `fork()`, и отличаются они только значением `child`: родитель видит в `child` номер дочернего процесса, а `child` видит там 0.

Далее идёт развилка, использующая переменную `child`: в ветке `child == 0` мы перенаправляем стандартные потоки ввода для `child` на необходимые нам трубы и файл:

- `dup2(pipe_data[0], STDIN_FILENO)` – меняю стандартный поток для ввода данных на трубу с данными от родителя

- `dup2(file_fd, STDOUT_FILENO)` - меняю стандартный поток для вывода данных на указанный файл

- `dup2(pipe_error[1], STDERR_FILENO)` - меняю стандартный поток ошибок на вход трубы для передачи ошибок

Так как `fork()` создаёт полную копию родительского процесса, в том числе и дескрипторы тоже, нам нужно закрыть все потоки, которые не будут использоваться дочерним процессом.

Эти закрываем, так как ребёнок их вообще не использует:

- `close(pipe_data[1])`

- `close(pipe_error[0])`

А эти закрываем, так как ранее мы перенаправили на них стандартные потоки, и теперь они просто не нужны копии:

- `close(pipe_data[0])`

- `close(file_fd)`

- `close(pipe_error[1])`

Запускаем дочернюю программу:

```
- int status = execv(SERVER_PROGRAM_PATH, args)
```

На ветке родителя:

Закрываем не используемые трубы и путь в файл (непосредственно родитель ничего в файл не пишет, это делает ребёнок):

```
- close(pipe_data[0])
```

```
- close(file_fd)
```

```
- close(pipe_error[1])
```

После этого родитель читает поток ввода пользователя и отправляет строки ребёнку на обработку (предполагаем, что пользователь вводит строки длины 4096 или меньше). После того, как пользователь закончит ввод, прочтёт и выведет полученные от ребёнка ошибки.

Программа ребёнка только читает строки, проверяет, является первая буква заглавной или нет, если да – пишет в стандартный поток ввода, иначе отправляет ошибку в стандартный поток для ошибок.

## Код программы

### parent.c

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <fcntl.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
#include <ctype.h>
```

```
// будем считать, что на вход не будут подаваться строки длины больше 4096
```

```
int read_line(char *buf, size_t size) // читает строку размера не более size из STDIN
```

```
{
```

```
    size_t n = 0;
```

```
    while (n < size - 1)
```

```
    {
```

```

    ssize_t r = read(STDIN_FILENO, &buf[n], 1);

    if (r == -1) return -1;

    if (r == 0) break;

    if (buf[n] == '\n') {n++; break;}

    n++;
}

buf[n] = '\0';

return (int)n;
}

int main(void)
{
    char line[4096];

    int n;

    // читаем и обрабатываем строки из ввода пока они не закончатся

    while ((n = read_line(line, sizeof(line))) > 0) { // читаем строку из ввода

        if (isupper((unsigned char)line[0])) { // проверяем первый символ строки, если он
заглавный - пишем в стандартный поток вывода

            write(STDOUT_FILENO, line, n);

        } else { // если нет, посылаем ошибки в STDERR

            size_t len = (size_t)n;

            if (len > 0 && line[len - 1] == '\n') len--; // \n в сообщение не включаем

            const char pre[] = "error: \n";

            const char post[] = "\n does not start with an uppercase letter\n";

            write(STDERR_FILENO, pre, strlen(pre));

            write(STDERR_FILENO, line, len);

            write(STDERR_FILENO, post, strlen(post));

        }

    }

    return 0;
}

```

```
}
```

## **child.c**

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <fcntl.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
#include <ctype.h>
```

```
// будем считать, что на вход не будут подаваться строки длины больше 4096
```

```
int read_line(char *buf, size_t size) // читает строку размера не более size из STDIN
```

```
{
```

```
    size_t n = 0;
```

```
    while (n < size - 1)
```

```
    {
```

```
        ssize_t r = read(STDIN_FILENO, &buf[n], 1);
```

```
        if (r == -1) return -1;
```

```
        if (r == 0) break;
```

```
        if (buf[n] == '\n') {n++; break;}
```

```
        n++;
```

```
    }
```

```
    buf[n] = '\0';
```

```
    return (int)n;
```

```
}
```

```
int main(void)
```

```
{
```

```

char line[4096];

int n;

// читаем и обрабатываем строки из ввода пока они не закончатся

while ((n = read_line(line, sizeof(line))) > 0) { // читаем строку из ввода

    if (isupper((unsigned char)line[0])) { // проверяем первый символ строки, если он
заглавный - пишем в стандартный поток вывода

        write(STDOUT_FILENO, line, n);

    } else { // если нет, посылаем ошибки в STDERR

        size_t len = (size_t)n;

        if (len > 0 && line[len - 1] == '\n') len--; // \n в сообщение не включаем

        const char pre[] = "error: \n";

        const char post[] = "\" does not start with an uppercase letter\n";

        write(STDERR_FILENO, pre, strlen(pre));

        write(STDERR_FILENO, line, len);

        write(STDERR_FILENO, post, strlen(post));

    }

}

return 0;

}

```

## Протокол работы программы

Ввод в программу:

test

Eee

Uuu

tt tt

p

Текст, оказавшийся в файле:

Eee

Uuu

Ошибки, выведенные после окончания ввода:

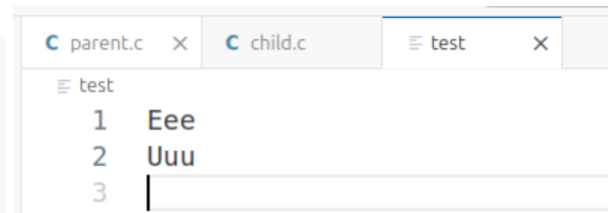
error: "tt tt" does not start with an uppercase letter

error: "p" does not start with an uppercase letter

Ввод в консоль

```
test
Eee
Uuu
tt tt
p
error: "tt tt" does not start with an uppercase letter
error: "p" does not start with an uppercase letter
```

Текстовый файл



The screenshot shows a text editor window with three tabs: 'parent.c', 'child.c', and 'test'. The 'test' tab is active and displays the following content:

```
test
1 Eee
2 Uuu
3 |
```

## Вывод

Было немного сложно в начале понять, почему в родительской программе мы пишем ветку для ребёнка, в которую как будто не должны иметь возможность зайти. Сильно запуталась с тем, что делает `dup2` и что после этого надо закрывать, в том числе из-за того, что трубы назвала `pipe1` и `pipe2`, после переименовала соответственно в `pipe_data` и `pipe_error`. В итоге со всем разобралась и лабораторная кажется теперь довольно интересной, а программа и процессы логичной и приятной структурой.